Multi-objective Diet Optimization Problem

Bioinspired Algorithms for Optimization

Group Project Assignment

Group identifier: PGN-XX

Members and contribution hours

Member	Hours
Ismael De Los Santos	XX h
Member 2 (Full Name)	XX h
Member 3 (Full Name)	XX h

The grade will be proportional to the implication of each member.

Universidad Politecnica de Madrid

May 14, 2026

Contents

Abstract

This work addresses the multi-objective weekly diet planning problem proposed in the Bioinspired Algorithms for Optimization (BAO) course. Given a database of 2,616 real food items and a user profile (daily caloric target plus age), the goal is to assemble a seven-day plan in which every day contains five meals — morning snack, breakfast, lunch, afternoon snack and dinner — with slot-specific food categories, while simultaneously (i) matching the daily caloric target as closely as possible and (ii) approximating the recommended macronutrient split of 50% carbohydrates, 27.5% fat and 22.5% protein. We formulate the task as a constrained two-objective minimisation problem on a length-77 decision vector and solve it with Pareto-based bioinspired metaheuristics: NSGA-II and PAES (evolutionary), plus MOPSO and P-ACO (swarm intelligence). We further compare two encodings (direct integer indices and random keys in $[0, 1]$) and three constraint-handling strategies (repair, penalty and death penalty). The final benchmark contains seven configurations, executed 30 times per user profile on all five profiles and assessed with hypervolume, IGD, Schott's spacing and Deb's Delta Spread. Statistical analysis follows the BAO chapter 6 protocol: Friedman Aligned Ranks for the omnibus and Wilcoxon signed-ranks with Shaffer's static post-hoc for pairwise comparisons.

1 Problem

1.1 Background

Why diet planning matters. Personalised nutrition planning is a recurring task in clinical dietetics, sports nutrition and consumer health applications. International guidelines (the World Health Organisation, the European Food Safety Authority and most national societies of nutrition) frame a healthy diet around two complementary axes:

1. **Energy balance.** The plan must meet a per-user daily caloric target whose value depends on age, sex, weight, height and physical activity. Sustained over- or under-shoot leads, respectively, to weight gain or unintentional caloric restriction.
2. **Macronutrient distribution.** Within that energy budget, the calories supplied by carbohydrates, fats and proteins should follow a recommended proportion. The literature converges around 50–60% carbohydrates, 25–30% fat and 10–35% protein. The course-specified target (50/27.5/22.5) sits inside that window and is the benchmark we optimise against.

Why this is hard. Producing a coherent plan by hand quickly becomes tedious: the food catalogue is large, every meal of the day expects a specific set of food types (alcohol does not appear at breakfast, sugar items belong only to snacks, eggs do not appear at lunch), and the two objectives conflict in non-trivial ways. Choosing a high-protein lunch helps the protein target but adds calories that must be balanced elsewhere; choosing a low-calorie carbohydrate-rich breakfast helps the carbohydrate target but may leave the day below its caloric goal.

Why bioinspired metaheuristics. From an optimisation perspective the problem is a constrained combinatorial selection problem with multiple conflicting objectives. The macronutrient objective is a non-linear ratio, the per-meal admissible food sets are structured and irregular, and we will see in §1.2 that the size of the search space is far beyond what enumeration or exact methods can reasonably handle. Bioinspired metaheuristics fit this setting well: they explore large discrete spaces efficiently without requiring gradient information, and the selected algorithms are designed or adapted for Pareto optimisation with several conflicting criteria.

1.2 Problem definition

The definition is built up in five steps.

(1) The planning unit. A solution is a *weekly meal plan* for one user, covering seven days $d \in \{1, \dots, 7\}$.

(2) Structure of one day. Each day contains five meals, broken down into slots that are each filled by one food from the catalogue:

- Morning snack: 1 slot (one fruit or sugar item).
- Breakfast: 3 slots (one breakfast drink and two breakfast foods).
- Lunch: 3 slots (one drink and two foods).
- Afternoon snack: 1 slot (one fruit or sugar item).
- Dinner: 3 slots (one drink and two foods).

Total: $1 + 3 + 3 + 1 + 3 = 11$ slots per day, $11 \times 7 = 77$ slots per weekly plan. Slots are indexed globally as $j \in \{1, \dots, 77\}$.

(3) Foods, food groups and per-slot admissible sets. The food catalogue $\mathcal{F} = \{1, \dots, 2616\}$ is the set of indices of the 2,616 foods in the course-provided database. Every food carries a hierarchical group prefix (`FA`, `MR`, `P`…) that encodes its category. For every slot j we pre-compute the *admissible set* $D_j \subseteq \mathcal{F}$ depending on the slot's role and on the user's age:

- **Snacks**: groups starting with `F` (fruits) or `S` (sugars).
- **Breakfast drinks**: `BA` (cow milk), `BH` (dairy drinks), `PA` (powdered drinks, infusions), `FE` (juices), `FC` (fruit-derived drinks).
- **Breakfast foods**: `A` (cereals), `C` (eggs), `FA` (general fruits), `MAA` (bacon), excluding `AC`, `AD`, `AE` (rice, pasta, pizza).
- **Lunch and dinner drinks**: `P` (general drinks), `FC`, `FE`, never `PA`; plus `Q` (alcohol) when edad ≥ 18.
- **Lunch and dinner foods**: any food whose prefix is *not* in $\{\texttt{FC}, \texttt{FE}, \texttt{P}, \texttt{Q}, \texttt{BA}, \texttt{BH}, \texttt{PA}, \texttt{S}, \texttt{A}\}$, plus the substantive cereals `AC, AD, AE, AF`.

A weekly plan is therefore the vector $\mathbf{x} \in \mathbb{N}^{77}$ in which x_j is the food index assigned to slot j. It is feasible iff $x_j \in D_j$ for every slot.

(4) Size of the search space. The number of feasible weekly plans is $N_{\text{plans}} = \prod_{j=1}^{77} |D_j|$. Per-slot $|D_j|$ ranges from a few dozen (snack slots) to roughly two thousand (lunch and dinner food slots), so N_{plans} is many orders of magnitude beyond any feasible enumeration. Exact methods such as integer linear programming would also struggle here because the macronutrient objective is a non-linear ratio.

(5) Objectives. For day d, let $K(\mathbf{x}, d)$, $C(\mathbf{x}, d)$, $F(\mathbf{x}, d)$ and $P(\mathbf{x}, d)$ denote the total calories and the grams of carbohydrate, fat and protein. Let T be the user's kcal/day target. Using the standard Atwater factors,

$$K_C^{(d)} = 4\,C(\mathbf{x}, d), \quad K_F^{(d)} = 9\,F(\mathbf{x}, d), \quad K_P^{(d)} = 4\,P(\mathbf{x}, d), \quad K_T^{(d)} = K_C^{(d)} + K_F^{(d)} + K_P^{(d)}.$$

The two minimisation objectives are

$$f_1(\mathbf{x}) = \sum_{d=1}^{7} \big| K(\mathbf{x}, d) - T \big|, \tag{1}$$

$$f_2(\mathbf{x}) = \sum_{d=1}^{7} \Big(|\, p_C^{(d)} - 50| + |\, p_F^{(d)} - 27.5| + |\, p_P^{(d)} - 22.5| \,\Big), \tag{2}$$

with $p_C^{(d)} = 100\,K_C^{(d)}/K_T^{(d)}$ and similarly for $p_F^{(d)}$, $p_P^{(d)}$. The full problem is

$$\min_{\mathbf{x}} \big(f_1(\mathbf{x}),\, f_2(\mathbf{x}) \big) \quad \text{subject to} \quad x_j \in D_j \;\; \forall j.$$

Note on the macronutrient targets. The course-specified percentages add to $50 + 27.5 + 22.5 = 100\%$. Therefore, the macronutrient objective in (2) has a theoretical optimum of zero when the daily macro-calorie proportions exactly match the target split.

2 Algorithm design

2.1 Metaheuristics used

Four metaheuristics span both course families (evolutionary algorithms and swarm intelligence). All four return Pareto archives/fronts and share the same feasibility logic, fitness function and metric pipeline so that any observed difference is attributable to the algorithm itself, not to plumbing.

- **NSGA-II** *(evolutionary, multi-objective)* [1]. Selection by non-dominated sorting plus crowding-distance preservation. The de-facto baseline for Pareto-based EAs.
- **PAES** *(evolutionary, multi-objective)* [2]. A $(1{+}1)$ evolution strategy paired with an adaptive grid archive. Contrasts NSGA-II both in cardinality (single working solution rather than a population) and in selection mechanism (archive-based rather than dominance-based), which makes it a useful complementary baseline.
- **MOPSO** *(swarm, multi-objective)* [3,5]. Particles move in random-key space and are guided by leaders from an external non-dominated archive. The primary leader rule is the Sigma method of Mostaghim and Teich; a random archive pick is available as a fallback. The archive is bounded with crowding-distance pruning.

- **P-ACO** *(swarm, multi-objective).* Ants construct direct-index weekly plans from per-slot pheromone distributions. A Pareto archive stores non-dominated plans, crowding-distance pruning maintains diversity, and archive members reinforce the pheromone trails without collapsing the objectives into a weighted sum.

2.2 Codifications used

We implement two encodings under a common interface (`Representation` in `diet_bao/representations`). NSGA-II, PAES and P-ACO use the discrete encoding, which matches the constructive meal-plan structure. NSGA-II and MOPSO additionally use the continuous random-key encoding; for MOPSO this is required because the velocity update is intrinsically continuous.

- **Direct-index encoding.** A length-77 integer vector; gene j is the food index assigned to slot j. Domain awareness is built into the operators: the random generator samples each gene uniformly from D_j, and the mutation operator either re-samples the gene from D_j or leaves it unchanged. Crossover and mutation therefore always produce feasible plans.
- **Random-key encoding.** A length-77 real vector with $k_j \in [0, 1]$. Decoding partitions $[0, 1]$ into $|D_j|$ equal-width bins and selects the food whose bin contains k_j:

$$\text{decode}(k_j, D_j) \;=\; D_j\big[\min\big(\,\lfloor k_j \cdot |D_j| \rfloor,\; |D_j| - 1\big)\big]\,.$$

  This converts the discrete search space into a continuous one while still guaranteeing feasibility. Mutation is Gaussian ($\sigma = 0.1$) with reflection; crossover (NSGA-II) is the default n-point operator from inspyred.

2.3 Implementation details

The project is written in Python 3.10+ and uses `inspyred` (bioinspired primitives), `mysql-connector-python` (database access), `numpy`/`pandas` (data manipulation), `scipy.stats` (statistical tests) and `matplotlib` (plots). The food database is restored from the course-provided MySQL dump and queried through the `diet_bao.data` module.

The codebase is organised around clear responsibilities, each in its own package:

- `representations/`: `direct_index` and `random_key` encodings behind a shared `Representation` protocol.
- `constraints/`: `repair`, `penalty` and `death_penalty` handlers behind a shared `ConstraintHandler` protocol.
- `metrics/`: pure-Python implementations of 2D hypervolume, IGD, Schott's spacing and Deb's Delta Spread.
- `ea/` and `si/`: the algorithm wrappers (`nsga2_diet`, `paes_diet`, `mopso_diet`, `paco_diet`).
- `experiment/`: benchmark composition (`AlgorithmConfig`, `BenchmarkPlan`, `run_all_subjects`) and visualisation helpers.
- `stac/`: Wilcoxon signed-ranks, Friedman, Friedman Aligned Ranks, Shaffer's post-hoc and Holm correction.

A suite of unit and integration tests is run with `pytest` before any experiment. Listing 1 shows the domain-aware mutation that keeps the direct-index encoding feasible at every generation.

Listing 1: Domain-aware mutation for direct-index NSGA-II.

```
def variator(random, candidates, args):
    rate = float(args.get("mutation_rate", 0.1))
    out = []
    for cand in candidates:
        mutant = list(cand)
        for i, domain in enumerate(state.per_position):
            if mutant[i] not in domain or random.random() < rate:
                mutant[i] = domain[random.randrange(len(domain))]
        out.append(mutant)
    return out
```

3 Experiments design

3.1 Parameter settings and tuning protocol

The notebook implements a small but deliberate **tuning sweep** on subjects 1 and 4 (one large and one small caloric target), selecting configurations by highest mean hypervolume and breaking ties by mean runtime. The intended grids are:

Algorithm	Swept dimensions	Candidate values
NSGA-II	pop, gen, mut. rate	$\{40, 80\} \times \{40, 80\} \times \{0.05, 0.1, 0.2\}$ (12 cells)
PAES	gen, archive size, mut. rate	$\{400, 800\} \times \{40, 80\} \times \{0.05, 0.1, 0.2\}$ (12 cells)
MOPSO	pop, gen, archive, acceleration	$\{30, 60\} \times \{40, 80\} \times \{40, 80\} \times \{1.5, 2.0\}$ (16 cells)
P-ACO	pop, gen, archive, evaporation	$\{30, 60\} \times \{40, 80\} \times \{40, 80\} \times \{0.05, 0.1\}$ (16 cells)

Table 1: Per-algorithm hyperparameter sweep. Every cell is replicated 5 times on each of 2 subjects. Selection criterion: highest mean hypervolume averaged across both subjects; ties broken by mean runtime.

The full tuning results, including the mean hypervolume, IGD and runtime per (algorithm, cell), are saved to `experiments/tuning_results.csv` and the picked winners to `experiments/tuning_best.csv`. The completed notebook run regenerated this file for all four algorithms; Table 2 therefore reports the actual tuning winners consumed by the full benchmark rather than fallback defaults.

Algorithm	Settings used	Source	Mean HV	Mean rt. (s)
NSGA-II	pop=80, gen=80, mut=0.2	tuning winner	969,654	0.99
PAES	pop=1, gen=800, archive=40, mut=0.1	tuning winner	474,575	1.39
MOPSO	pop=30, gen=80, archive=40, $w = 0.4$, $c_1 = c_2 = 2.0$	tuning winner	489,401	0.21
P-ACO	pop=60, gen=40, archive=40, $\rho = 0.1$, $\alpha = 1$, $\tau_0 = 1$	tuning winner	459,679	16.39

Table 2: Hyperparameter settings used by the full benchmark in `experiments/all_runs.csv`. The mean hypervolume and runtime columns come from `experiments/tuning_best.csv`.

The seven main-grid configurations (Table 3) consume the settings in Table 2. PAES uses a $(1+1)$-ES so its population is 1 by construction; we compensate with a larger generation budget.

Config id	Algorithm	Encoding	Pop	Gen	Notes
`nsga2_di_repair`	NSGA-II	direct_index	80	80	repair handler, mut. rate 0.2
`nsga2_di_penalty`	NSGA-II	direct_index	80	80	penalty $(1000, 10)$ per violation
`nsga2_di_death`	NSGA-II	direct_index	80	80	infeasible $\to +\infty$
`nsga2_rk_repair`	NSGA-II	random_key	80	80	Gaussian mutation $\sigma = 0.1$
`paes_di_repair`	PAES	direct_index	1	800	archive size 40, mut. rate 0.1
`mopso_rk`	MOPSO	random_key	30	80	Sigma leader, archive 40
`paco_di`	P-ACO	direct_index	60	40	per-slot pheromones, archive 40

Table 3: Seven main-grid configurations used in the full experiment.

3.2 Experimental setup

Each of the seven configurations is executed **30 stochastic replicates** on each of the **five user profiles**, totalling $7 \times 5 \times 30 = 1050$ runs. Per-replicate seeds are deterministic and unique (`seed0+r`), so the experiment is fully reproducible. The notebook `main.ipynb` schedules every (subject, configuration, seed) triple through `concurrent.futures.ProcessPoolExecutor` (`BenchmarkPlan(n_jobs=4)` by default) and writes the raw rows to `experiments/all_runs.csv`.

Metrics. For every run we record the best f_1, the best f_2, the runtime and the size of the Pareto front. From the raw front we additionally compute four multi-objective indicators:

- **Hypervolume (HV).** 2D area dominated by the front relative to a per-front reference point set adaptively at $1.1\times$ the front maxima plus a small offset. Higher is better.
- **Inverted generational distance (IGD).** Mean Euclidean distance from each point of a per-subject reference set (the union of all fronts produced by all algorithms on that subject) to the nearest point of the obtained front. Lower is better.
- **Schott's spacing.** Standard deviation of consecutive Euclidean distances along the front. Lower is more uniform.
- **Delta Spread (Deb's Δ).** The diversity metric taught in BAO chapter 6.2. Combines the extent of the front (distance from each obtained extreme to the corresponding reference extreme) with the uniformity of internal spacing: $\Delta = (\sum_j d_j^e + \sum_i |d_i - \bar{d}|)/(\sum_j d_j^e + |S|\,\bar{d})$. Lower is better.

Statistical tests. Following the procedure recommended in BAO chapter 6 ("Comparing Algorithms Output"), our 30 replicates are paired across configurations because they share the same seed schedule. We therefore use the two-sided **Wilcoxon signed-ranks test** [9] for 1-vs-1 paired comparisons rather than the unpaired Mann–Whitney U. The omnibus across all seven configurations uses the **Friedman Aligned Ranks test** [8], a more powerful variant of the plain Friedman test that ranks all block-aligned observations together. The N-vs-N post-hoc analysis is pairwise Wilcoxon with **Shaffer's static procedure** [11]; the simpler Holm–Bonferroni correction [10] is also reported for backward comparability.

Limitations. (i) A single dataset (the course's NutritionPlanning database) is used; results may not transfer to other food catalogues. (ii) The variation operators of the direct-index encoding produce only feasible candidates by construction, which makes the constraint-handler comparison degenerate (see §4). (iii) The tuning sweep in §3.1 is small (12–16 cells per algorithm) and is run on only two of the five subjects; a denser, full-subject sweep is a natural follow-up. (iv) The preference, dislike and allergy fields are loaded from the database but are not yet incorporated into the objective or constraints.

3.3 Data description

The MySQL dump provided by the course contains **2,616 food items**, each annotated with a hierarchical food-group prefix and per-item macronutrient content (kcal, carbohydrates, fat and protein, in grams per serving). It also contains **five user profiles** (ids 1–5) covering ages 17, 30, 40, 55 and 72 with caloric targets ranging from 1,401 to 3,103 kcal/day. Each profile additionally carries preference, dislike and allergy lists; these are loaded but not used in the present optimisation. All 2,616 foods participate in the experiments, and all five subjects are used for the comparison.

4 Experimental results

Result source. The numerical tables and figures in this section are read from the full-run artefacts generated by `main.ipynb`: `experiments/all_runs.csv` contains 1050 rows, and the notebook sanity checks confirm 7 configurations, 5 subjects and 30 seeds per pair.

4.1 Aggregated metrics

Table 4 reports the mean of every metric across the 30 replicates and 5 subjects per configuration ($n = 150$ per row), computed from `experiments/all_runs.csv` produced by `main.ipynb`. The three direct-index NSGA-II handlers are identical on every quality metric because the domain-aware variator never produces an infeasible candidate (see Discussion); runtime differs because the handlers have different evaluation overhead.

Config	HV ↑	IGD ↓	Spacing ↓	Δ-Spread ↓	f_1 best ↓	f_2 best ↓	Runtime (s) ↓	Front
`nsga2_rk_repair`	**1,059,760**	**120.20**	104.68	0.830	**1,105.10**	167.11	1.05	**80.00**
`nsga2_di_repair`	1,014,630	171.75	**97.71**	0.822	1,343.37	167.77	1.00	**80.00**
`nsga2_di_penalty`	1,014,630	171.75	**97.71**	0.822	1,343.37	167.77	1.97	**80.00**
`nsga2_di_death`	1,014,630	171.75	**97.71**	0.822	1,343.37	167.77	1.97	**80.00**
`paes_di_repair`	401,023	438.80	185.66	1.087	1,987.88	**123.92**	1.40	38.89
`mopso_rk`	402,911	640.19	330.53	0.937	2,362.08	175.83	**0.26**	17.14
`paco_di`	435,310	783.09	438.22	**0.775**	2,700.35	179.80	25.02	9.13

Table 4: Mean metrics per configuration across all 30 replicates and 5 subjects ($n = 150$ per row). Bold marks the best value in each column (ties are bolded). Higher is better for hypervolume; lower is better for IGD, spacing, Δ-Spread, f_1, f_2 and runtime. "Front" is the mean cardinality of the final non-dominated set.

4.2 Statistical significance

The **Friedman Aligned Ranks** omnibus across the seven main configurations is highly significant on every metric (the p-values underflow to 0.0 at machine precision in `experiments/friedman_aligned.csv`). Table 5 reports the average aligned ranks per configuration on three representative metrics.

Config	HV rank ↓	IGD rank ↓	Runtime rank ↓
`nsga2_di_death`	**310.19**	340.09	732.20
`nsga2_di_penalty`	**310.19**	340.09	737.65
`nsga2_di_repair`	**310.19**	340.09	245.90
`nsga2_rk_repair`	331.09	**260.57**	370.67
`paco_di`	791.20	904.60	975.50
`paes_di_repair`	805.40	654.05	521.61
`mopso_rk`	820.23	839.03	**94.97**

Table 5: Friedman *Aligned* Ranks per metric (lower is better). For hypervolume, the aligned observations are sign-flipped so that better HV produces better (lower) ranks. Values are read directly from `experiments/friedman_aligned.csv`.

The pairwise **Wilcoxon signed-ranks test with Shaffer's static post-hoc** on hypervolume (`experiments/pairwise_hv_shaf` yields the following picture:

- **NSGA-II vs. non-NSGA-II:** every NSGA-II variant is significantly better on HV than PAES, MOPSO and P-ACO. The largest adjusted p-value among these cross-family comparisons is still only 2.86×10^{-23}.
- **Within NSGA-II:** the three direct-index constraint handlers are exactly tied on HV ($p_{\text{adj}} = 1.0$). The random-key NSGA-II row has the highest mean HV and the best IGD, but its HV difference from the direct-index rows is not significant at $\alpha = 0.05$ ($p_{\text{adj}} \approx 0.359$).
- **Among non-NSGA-II methods:** the HV differences among MOPSO, PAES and P-ACO are not significant after Shaffer correction: MOPSO vs. PAES has $p_{\text{adj}} \approx 0.878$, MOPSO vs. P-ACO has $p_{\text{adj}} \approx 0.176$, and P-ACO vs. PAES has $p_{\text{adj}} \approx 0.193$.

4.3 Visualisations

Figure 1 contrasts the seven configurations on a single run for subject 1: the left panel shows their final Pareto fronts, the middle panel tracks the best scalar value over generations (convergence), and the right panel tracks front or archive size as a proxy for diversity. Figure 2 summarises the distribution of every metric across the per-configuration sample. Figure 3 clouds the best (f_1, f_2) point of every run for subject 1.

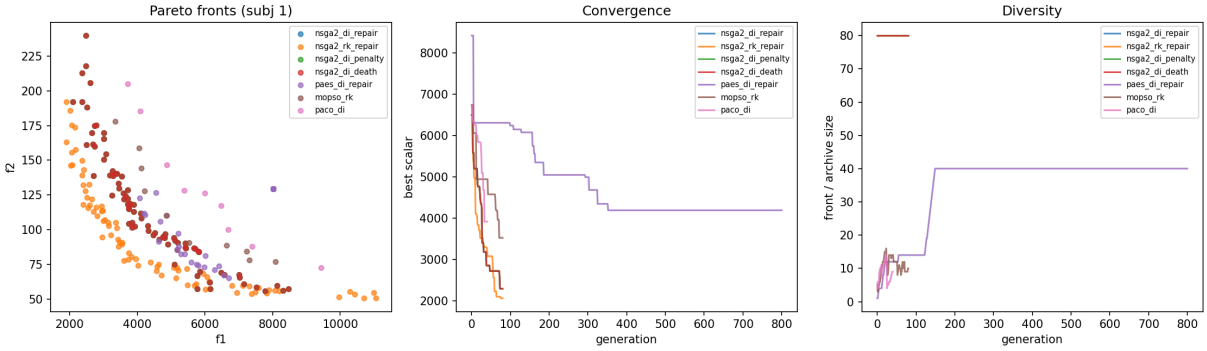

Figure 1: Single-run overview for subject 1: objective space (left), convergence (centre), diversity (right). NSGA-II maintains full 80-point fronts; PAES and MOPSO keep medium-sized archives; P-ACO returns a smaller pheromone-reinforced archive.

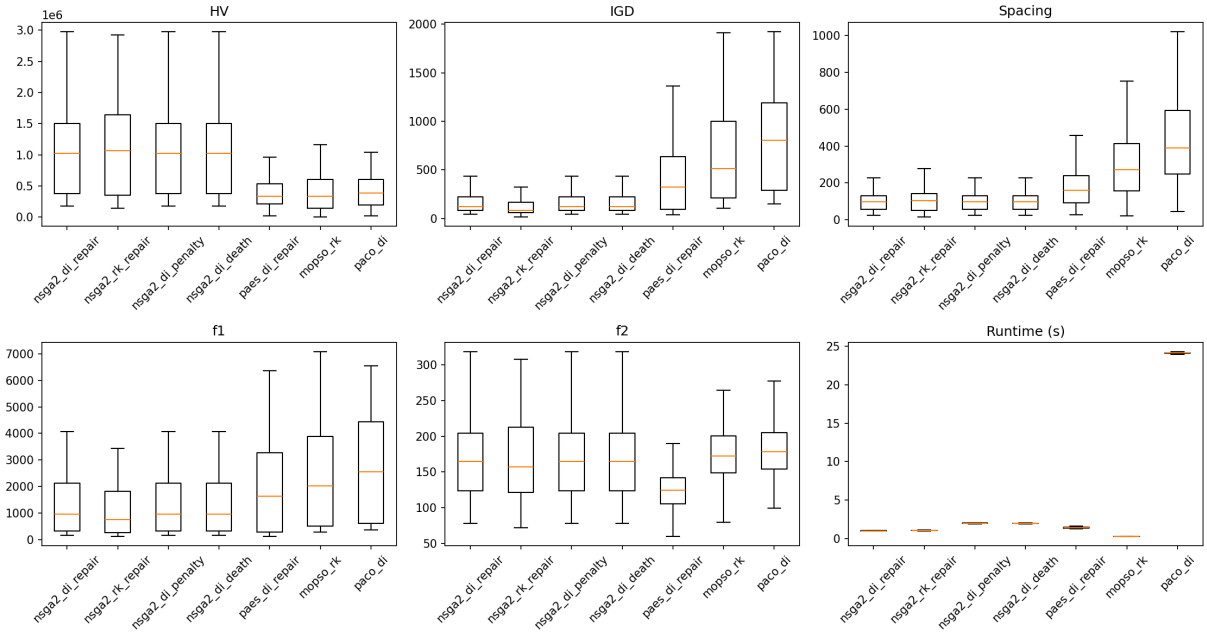

Figure 2: Per-configuration distributions across all 30 replicates $\times$ 5 subjects. Top row: hypervolume, IGD, spacing. Bottom row: best f_1, best f_2, runtime in seconds. NSGA-II variants dominate hypervolume and IGD; MOPSO is fastest; P-ACO is the slowest method in the current implementation.

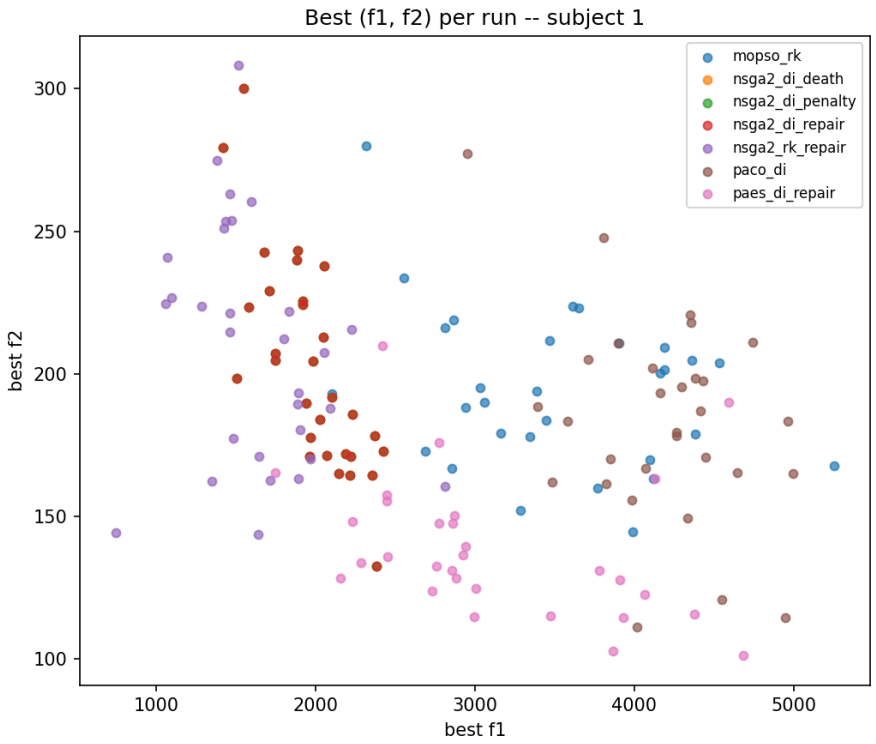

Figure 3: Cloud of best (f_1, f_2) values per run for subject 1, coloured by configuration. NSGA-II variants occupy the strongest calorie-error region, while PAES often supplies the strongest macronutrient-error points despite weaker Pareto coverage.

4.4 Discussion

NSGA-II is the dominant algorithm family on Pareto coverage. The random-key NSGA-II configuration achieves both the highest mean hypervolume ($1{,}059{,}760$) and the best mean IGD (120.20), while the three direct-index NSGA-II configurations remain close behind on hypervolume ($1{,}014{,}630$). All non-NSGA-II configurations are significantly worse than NSGA-II on hypervolume under Wilcoxon+Shaffer.

Random-key NSGA-II improves coverage without a significant HV gap. The hypervolume difference between `nsga2_di_repair` and `nsga2_rk_repair` is not significant at $\alpha = 0.05$ ($p_{\text{adj}} \approx 0.359$ on HV), but the mean hypervolume, IGD mean and aligned IGD rank all favour the random-key variant (Table 5). Runtime is similar for the two repair variants: 1.00 s for direct-index repair and 1.05 s for random-key repair.

Constraint handlers are equivalent in our setup. The three direct-index configurations (`repair`, `penalty`, `death_penalty`) produce identical hypervolume, IGD, spacing, f_1 and f_2 (Table 4), with $p_{\text{adj}} = 1.0$ in the Wilcoxon+Shaffer pairwise test on hypervolume. The reason is structural rather than statistical: our domain-aware mutation always picks each gene from D_j, so no infeasible candidate is ever produced and no handler changes the search trajectory. The only observable difference is runtime: `repair` is faster (1.00 s) than `penalty` and `death_penalty` (both 1.97 s). Meaningfully differentiating the three handlers would require an unrestricted variator (an item we list under Future Work).

PAES has the best per-point macronutrient quality but weaker Pareto coverage. The mean best f_2 for PAES is 123.92, the lowest of the seven configurations (Table 4). Hypervolume, however, is substantially lower than NSGA-II. This is an expected trade-off for a $(1{+}1)$-ES with grid archiving: it exploits one trajectory deeply but does not actively diversify the front as strongly as NSGA-II.

The swarm algorithms show different strengths. MOPSO is the fastest method (0.26 s), but its mean hypervolume ($402{,}911$) remains far below NSGA-II. P-ACO has the highest non-NSGA-II mean hypervolume ($435{,}310$) and the best mean Δ-Spread (0.775), but it is also the slowest method (25.02 s) and returns the smallest

archive on average (9.13 points). The non-NSGA-II hypervolume differences are not statistically significant after Shaffer correction.

4.5 Code repository

The full source code, tests and experiment notebook required to reproduce every figure and table is available in the accompanying repository.

The reproducibility workflow is: run `main.ipynb` end-to-end (database connection required), then execute `python experiments/emit_latex.py`, which prints the LaTeX-ready rows for the summary table, the Friedman aligned ranks and the Wilcoxon+Shaffer p-values that should be pasted into this document.

5 Conclusion and future work

This project addressed the multi-objective diet optimisation problem with two evolutionary algorithms (NSGA-II, PAES), two swarm intelligence algorithms (MOPSO, P-ACO), two encodings (direct-index, random-key) and three constraint handlers (repair, penalty, death penalty). The current implementation uses Pareto archives throughout: the previous weighted-sum swarm baselines were replaced by MOPSO and P-ACO so that the swarm side also optimises the actual multi-objective formulation. The full run completed 1050 executions and shows NSGA-II as the strongest family on Pareto coverage, MOPSO as the fastest method, PAES as the strongest on the macronutrient objective, and P-ACO as the strongest method on Δ-Spread.

A side finding that turned out to be educationally relevant is that the three constraint handlers became indistinguishable: because mutation always picks genes from valid domains, no infeasible candidate is ever produced and the handlers never fire. We treat this as a documented limitation of the variation operator and as motivation for the most natural follow-up.

Future work

Two advanced techniques covered in BAO chapter 7 were implemented as optional extensions in our codebase:

1. Parallelism (highly recommended, low effort). The main experimental grid is planned as 7 configs $\times$ 5 subjects $\times$ 30 replicates $= 1050$ independent algorithm executions, which makes it embarrassingly parallel. We therefore added a multiprocessing execution path to `run_all_subjects` using `concurrent.futures.ProcessPoolExecu` enabled by setting `BenchmarkPlan(n_jobs>1)`. This does not change per-run runtime measurements (each run is still timed individually), but it significantly reduces the wall-clock time required to complete the full grid.

2. Memetic algorithms / local search (highly recommended, moderate effort). To explore whether local exploitation can further improve the NSGA-II fronts, we implemented a simple memetic NSGA-II variant by injecting a 1-step, domain-respecting local search into the NSGA-II variator, controlled by a `memetic_rate` hyperparameter. The operator proposes a single-slot in-domain perturbation and accepts it when it Pareto-dominates (or improves the scalarised sum as a tie-break). This variant is available for additional experiments, but it is not part of the seven-configuration main grid so that Tables 3 and 4 remain directly comparable.

Additional smaller follow-ups. (i) Replace the domain-aware variator with a naive (unrestricted) one so the three constraint handlers can be properly differentiated. (ii) Extend the fitness function to incorporate the user-profile preferences, dislikes and allergies that the database carries but the present formulation ignores. (iii) Expand the tuning sweep to all five subjects and a denser grid, especially for MOPSO and P-ACO.

References

1. Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002). *A fast and elitist multiobjective genetic algorithm: NSGA-II*. IEEE Transactions on Evolutionary Computation, 6(2), 182-197.

2. Knowles, J., & Corne, D. (2000). *Approximating the nondominated front using the Pareto archived evolution strategy*. Evolutionary Computation, 8(2), 149-172.

3. Coello, C. A. C., Pulido, G. T., & Lechuga, M. S. (2004). *Handling multiple objectives with particle swarm optimization*. IEEE Transactions on Evolutionary Computation, 8(3), 256-279.

4. Nebro, A. J., Durillo, J. J., Garcia-Nieto, J., Coello Coello, C. A., Luna, F., & Alba, E. (2009). *SMPSO: A new PSO-based metaheuristic for multi-objective optimization*. IEEE Symposium on Computational Intelligence in Multi-Criteria Decision-Making, 66-73.

5. Kennedy, J., & Eberhart, R. (1995). *Particle swarm optimization*. Proceedings of ICNN'95, vol. IV, 1942-1948.

6. Schott, J. R. (1995). *Fault tolerant design using single and multicriteria genetic algorithm optimization*. Master's thesis, Massachusetts Institute of Technology.

7. Zitzler, E., & Thiele, L. (1999). *Multiobjective evolutionary algorithms: A comparative case study and the strength Pareto approach*. IEEE Transactions on Evolutionary Computation, 3(4), 257-271.

8. Friedman, M. (1937). *The use of ranks to avoid the assumption of normality implicit in the analysis of variance*. Journal of the American Statistical Association, 32(200), 675-701.

9. Wilcoxon, F. (1945). *Individual comparisons by ranking methods*. Biometrics Bulletin, 1(6), 80-83.

10. Holm, S. (1979). *A simple sequentially rejective multiple test procedure*. Scandinavian Journal of Statistics, 6(2), 65-70.

11. Shaffer, J. P. (1986). *Modified sequentially rejective multiple test procedures*. Journal of the American Statistical Association, 81(395), 826-831.

12. Garrett, A. (2012). *Inspyred: Bio-inspired algorithms in Python*. `https://aarongarrett.github.io/inspyred/`

13. Quesada Pajares, J. (2024). *NutritionPlanning: food database and reference scripts for the BAO course*. `https://github.com/javierquesadapajares/NutritionPlanning`

14. Coello, C. A. C., Lamont, G. B., & Van Veldhuizen, D. A. (2007). *Evolutionary Algorithms for Solving Multi-Objective Problems* (2nd ed.). Springer.